N.Adi Narayana

192525346

CSA0821

PYTHON PROGRAMMING

Keerthana S

Python-Based Milk Collection and Farmer Payment Management System

The aim of this project is to create a simple Python program to calculate the payment for milk supplied by farmers. The program takes milk quantity and rate per litre as input and calculates the total amount. It also accepts milk quality details like fat and SNF percentages. This helps save time and reduces mistakes in manual calculations.

Problem Statement

In villages, milk farmers supply milk to co-operative societies every day. Calculating the payment manually can take time and may lead to mistakes. It is difficult to calculate the correct amount when many farmers supply milk.To solve this problem, we are developing a Python-based Milk Collection and Farmer Payment Management System. It takes the farmer's details, milk quantity, fat percentage, SNF percentage, and rate per litre as input. It calculates the total payment and displays the result.This system helps save time, reduce calculation mistakes, and make the payment process easier for farmers and milk society staff.

Objectives of the Project

- To calculate the payment for milk supplied by farmers.
- To collect milk quantity and quality details.
- To reduce mistakes in manual calculations.
- To display the total payment clearly.
- To save time and make the payment process easier.

Proposed Solution

We are developing a simple Python program to calculate the payment for milk supplied by farmers. The program takes the farmer's name, milk quantity, fat percentage, SNF percentage, and rate per litre as input. It calculates the total payment automatically and displays the result. This makes the calculation process easy, saves time, and reduces manual mistakes.

Hardware and Software Requirements

Hardware Requirements:

1. Computer or Laptop
2. Minimum 4 GB RAM
3. Keyboard and Monitor

Software Requirements:

1. Operating System: Windows, Linux, or macOS
2. Programming Language: Python
3. Code Editor: IDLE or Visual Studio Code

Python Concepts Used

1. **Variables:** To store farmer details and milk quantity.
2. **Data Types:** To store names and numerical values.
3. **Operators:** To calculate the total payment.
4. **User Input:** To collect details from the user.
5. **Conditional Statements:** To check whether the entered values are valid.
6. **Functions:** To make the program simple and reusable.
7. **Output Statements:** To display the payment details.

TASK 1 – PROBLEM ANALYSIS

1. Problem Statement

The aim of this project is to develop a Python-based Milk Payment Calculation System for a village milk co-operative. The system calculates the payment for farmers based on the quantity of milk supplied, fat percentage, and SNF percentage. It calculates the rate per litre, individual payment, and total payment for ten days. This system helps reduce manual errors and makes the payment process faster and more reliable.

2. Inputs and Outputs

Inputs

Input	Data Type	Purpose
Farmer ID	String	Identifies each farmer
Milk Quantity	Decimal	Stores the quantity of milk supplied in litres
Fat Percentage	Decimal	Represents the fat content of the milk
SNF Percentage	Decimal	Represents the solids-not-fat content
Shift	String	Identifies the morning or evening collection

Outputs

Output	Data Type	Purpose
Rate per Litre	Decimal	Displays the calculated price per litre
Payment Amount	Decimal	Displays the payment for each milk entry
Ten-Day Total	Decimal	Displays the total payment for ten days

3. Constants

The following constants are used in the milk payment calculation:

- BASE_RATE = 30.00
- FAT_STEP_VALUE = 0.40
- SNF_STEP_VALUE = 0.30

- STANDARD_FAT = 3.5
- STANDARD_SNF = 8.5

These constants represent the basic rate and milk quality adjustment values. They can be updated when the milk co-operative changes its payment rules.

4. Validation Rules

- Milk quantity must not be negative.
- Fat percentage must be between 2.0% and 7.0%, inclusive.
- SNF percentage must be within the range approved by the milk co-operative.
- Farmer ID must not be empty.
- The collection shift must be valid.
- Payment must be calculated using the approved rate card and rounding rules.

5. Alternative Design

One approach is to create separate variables for every farmer. However, this becomes difficult to manage when the number of farmers increases.

A better approach is to store each farmer's details in a list of tuples. Each record contains the farmer ID, milk quantity, fat percentage, SNF percentage, and shift. A loop can process these records efficiently.

Time Complexity: O(n), because each record is processed once.

Space Complexity: O(n), because all records are stored in a list.

6. Conclusion

The proposed Milk Payment Calculation System uses suitable data types, constants, input validation, and a list-based structure. It helps calculate farmer payments clearly and consistently, reduces manual calculation errors, and makes the program easier to maintain and improve.

TASK-2

1. Aim

To analyse Python data types, arithmetic operators, floating-point precision, and different methods of handling money accurately in a milk payment system.

2. Introduction

Python supports different data types, such as integers, floating-point numbers, strings, Booleans, lists, and Decimal values. These types behave differently during calculations. In a milk payment system, understanding these differences is important because even a small rounding error can affect the total payment when many transactions are processed.

3. Python Interpreter – Values and Types

The following expressions demonstrate the types of different Python values:

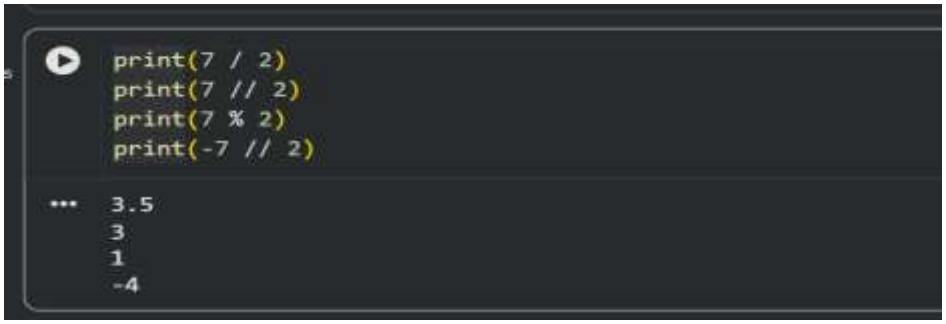


Explanation:

- int stores whole numbers.
- float stores numbers with decimal points.
- str stores text.
- bool represents True or False.

4. Arithmetic Operators

Python provides different operators for division and remainder calculations.

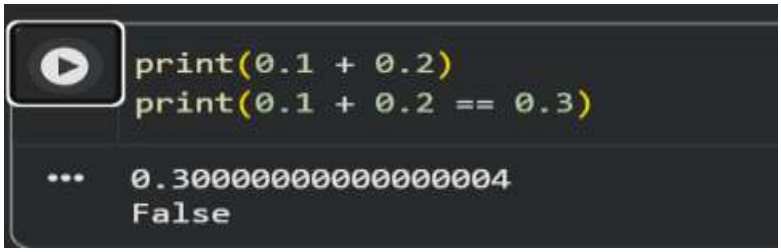


Explanation:

- / performs normal division and returns 3.5.
- // performs floor division and returns 3.
- % returns the remainder, which is 1.
- -7 // 2 returns -4 because floor division rounds down towards negative infinity.

5. Floating-Point Precision

Python's float type represents numbers using binary floating-point arithmetic. Some decimal fractions cannot be represented exactly in binary.



Explanation:

The result is slightly different from 0.3 because of floating-point representation. Therefore, directly comparing certain decimal calculations with == can produce unexpected results.

6. Complete the Code – Float and Decimal Comparison

The following program compares float and Decimal calculations for 1,000 payments of ₹33.40 and three adjustments of ₹0.10.

```
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3
def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total
def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
print("Float:", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

```
Float: 33400.30000000061
Decimal: 33400.30
False
True
```

Note: The exact float output may vary slightly depending on the calculation and Python environment.

7. Why Decimal Is Preferred for Money

The Decimal type supports decimal arithmetic and predictable rounding. When values are created from strings, the decimal values are represented as intended.

Complexity Analysis:

- Time Complexity: O(n), because each payment is processed once.
- Additional Space Complexity: O(1) for the running total, excluding the input list.
- Input Storage: O(n), because the payment values are stored in a list.

9. Evidence Expected

The evidence for this task includes:

- An interactive Python session showing the types of values.
- Results of division, floor division, and modulus operators.
- The floating-point comparison result.

10. Conclusion

This task demonstrates how Python data types and arithmetic operators affect calculations. Floating-point values can introduce small precision errors, which may accumulate across many transactions. Integer paise and Decimal provide more reliable alternatives for monetary calculations. For the milk payment system, Decimal created from strings is recommended for accurate and consistent financial results.

TASK 3 – CHOOSE VARIABLES AND NAME THE CONSTANTS

1. Aim

To define meaningful variable names and named constants for the milk payment system so that rate changes can be managed easily in one place.

2. Analysis

The milk payment program uses variables to store changing values such as milk quantity, fat percentage, SNF percentage, and calculated payment. Constants are used for fixed rate-card values, including the base rate and fat and SNF adjustment values.

Python does not enforce constants directly. Uppercase names are used by convention to indicate values that should not be changed during normal program execution.

3. Python Program

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30

farmer_id = 101
milk_quantity = 10.5
fat = 4.0
snf = 8.5

rate = BASE_RATE + (fat - 3.5) * FAT_RATE + (snf - 8.5) * SNF_RATE

print("Farmer ID:", farmer_id)
print("Milk Quantity:", milk_quantity)
print("Fat:", fat)
print("SNF:", snf)
print("Rate per Litre:", round(rate, 2))
```

```
Farmer ID: 101
Milk Quantity: 10.5
Fat: 4.0
SNF: 8.5
Rate per Litre: 30.2
```

5. Explanation

Variable naming: Names such as litres, fat, and snf describe the stored values clearly. Python variable names should be meaningful and follow naming conventions.

Named constants: Names such as BASE_RATE and FAT_STEP_VALUE use uppercase letters. If the co-operative revises a rate, the corresponding constant can be changed in one place.

Dynamic typing: Python allows a variable to store different data types at different times.

7. Result

Named constants and meaningful variables were used to organise the milk payment calculation. Dynamic typing, multiple assignment, and dictionary-based rate storage were also demonstrated. This approach makes the program easier to understand, maintain, and update.

TASK 4 – WRITE CORRECT EXPRESSIONS (4 MARKS)

1. Aim

To write correct Python expressions to calculate fat steps, SNF steps, milk rate per litre, and total payment. To understand operator precedence, rounding methods, floating-point errors, and comparison operators**.**

2. Python Code

```
BASE_RATE = 30.00
FAT_STEP = 0.40
SNF_STEP = 0.30

litres = 10.0
fat = 4.2
snf = 8.7

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = BASE_RATE + fat_steps * FAT_STEP + snf_steps * SNF_STEP
rate = round(rate, 2)

amount = round(litres * rate, 2)

print("Fat Steps:", fat_steps)
print("SNF Steps:", snf_steps)
print(f"Milk Rate: Rs. {rate:.2f}")
print(f"Total Payment: Rs. {amount:.2f}")

if 2.0 <= fat <= 7.0 and snf > 0:
    print("Valid Milk Quality")
else:
    print("Invalid Milk Quality")
```

```
input
SNF Steps: 2
Milk Rate: Rs. 33.40
Total Payment: Rs. 334.00
Valid Milk Quality
```

5. Explanation / Matter

1. Fat Steps

The expression (fat - 3.5) / 0.1 calculates the number of fat steps. The round() function converts the result into the nearest integer.

2. SNF Steps

The expression (snf - 8.5) / 0.1 calculates the number of SNF steps.

3. Milk Rate

The base rate is added to the fat and SNF adjustments to calculate the final milk rate.

Formula:

Milk Rate = Base Rate + (Fat Steps × Fat Step Value) + (SNF Steps × SNF Step Value)

= 30 + (7 × 0.40) + (2 × 0.30)

= ₹33.40 per litre

4. Total Payment

Total Payment = Milk Quantity × Milk Rate

= 10 × 33.40

= ₹334.00

6. Alternative Concept and Complexity Analysis

The number of fat steps can be calculated using round(), int(), or math.floor().

For fat = 4.2, all three methods produce 7 steps and a rate of ₹32.80 when SNF is at its standard value.

For fat = 3.1, round() and math.floor() produce -4 steps and a rate of ₹28.40, while int() produces -3 steps and a rate of ₹28.80.

Operation	Time Complexity	Space Complexity
Fat step calculation	O(1)	O(1)
SNF step calculation	O(1)	O(1)
Milk rate calculation	O(1)	O(1)
Payment calculation	O(1)	O(1)

The program performs a fixed number of operations. Therefore, the time complexity is O(1) and the space complexity is O(1).

7. Conclusion

Correct Python expressions help calculate milk rates and payments accurately. Understanding operator precedence, floating-point precision, rounding methods, and comparison operators helps prevent incorrect results. For financial calculations, Decimal is recommended with a clearly defined rounding policy.

TASK 5 – READ INPUT AND CONVERT TYPES (3 MARKS)

1. Aim

To read the clerk's milk collection details, convert input values into appropriate data types, validate the inputs, and display the milk rate and payment amount in a properly formatted manner.

2. Python Code

```
BASE_RATE = 30.00
FAT_STEP = 0.40
SNF_STEP = 0.30

try:
    litres = float(input("Enter milk quantity: "))
    fat = float(input("Enter fat percentage: "))
    snf = float(input("Enter SNF percentage: "))

    if litres <= 0:
        print("Invalid milk quantity")
    elif not (2.0 <= fat <= 7.0):
        print("Invalid fat percentage")
    elif snf <= 0:
        print("Invalid SNF percentage")
    else:
        fat_steps = round((fat - 3.5) / 0.1)
        snf_steps = round((snf - 8.5) / 0.1)

        rate = BASE_RATE + fat_steps * FAT_STEP + snf_steps * SNF_STEP
        rate = round(rate, 2)
        amount = round(litres * rate, 2)

        print("\nMILK PAYMENT DETAILS")
        print(f"Litres: {litres:8.2f}")
        print(f"Fat:    {fat:8.2f}")
```

input

```
Fat:         4.20
SNF:         2.00
Rate:   Rs.    13.30
Amount: Rs.   133.00
```

Input and Type Conversion: The input() function reads values as strings. We use float() to convert them into decimal numbers.

2. Input Validation: The try-except block handles invalid inputs like "eight". We also check whether the milk quantity, fat, and SNF values are valid.

3. Formatted Output: F-strings display the milk quantity, fat, SNF, rate, and payment neatly with two decimal places.

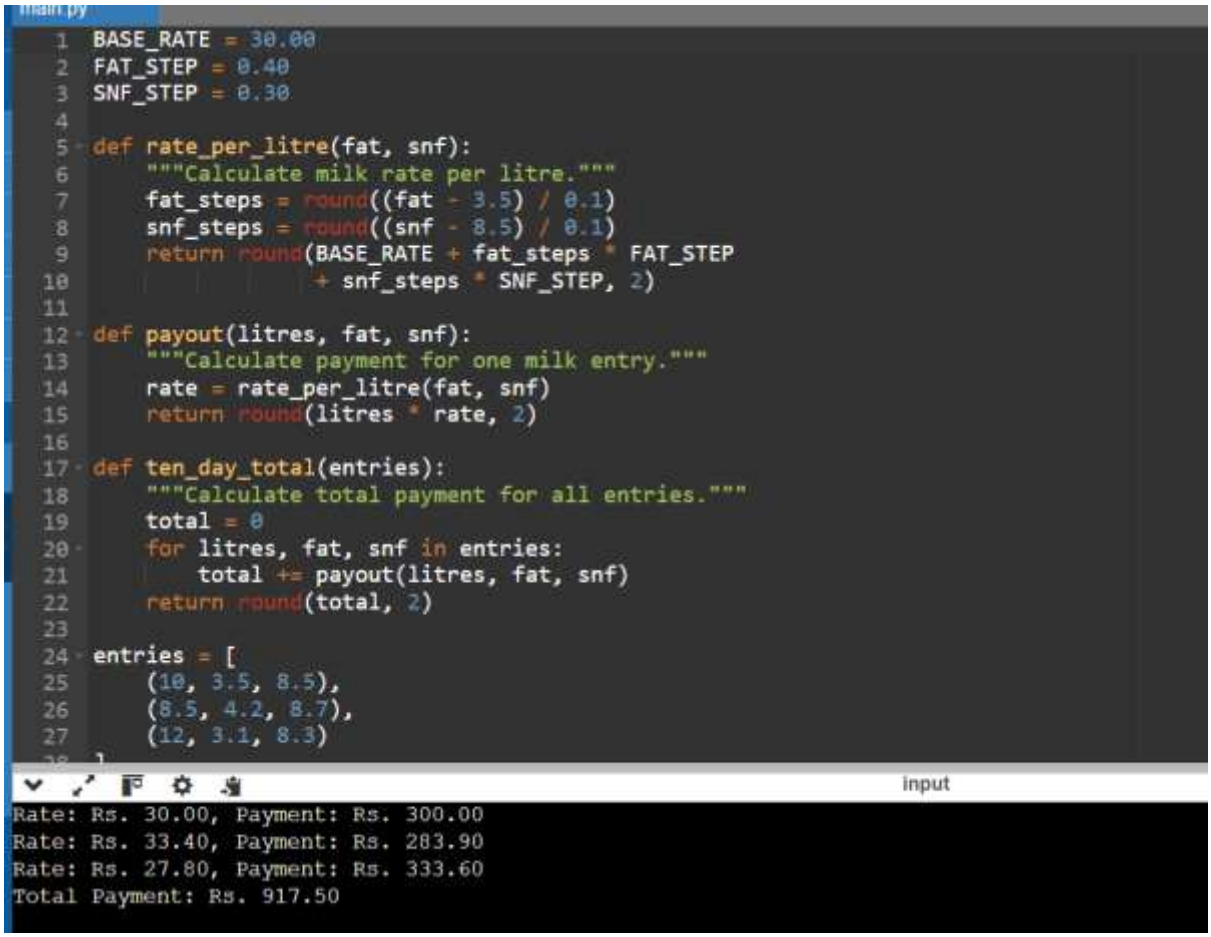
Conclusion: The program reads milk details, validates the inputs, calculates the payment, and displays the result clearly.

TASK 6 – ORGANISE THE LOGIC INTO FUNCTIONS (4 MARKS)

1. Aim

To divide the milk payment calculation into separate Python functions to make the program simple, reusable, and easy to understand.

2. Python Code



5. Explanation (Simple Words)

1. Functions: rate_per_litre() calculates the milk rate, payout() calculates the payment, and ten_day_total() calculates the total payment.
2. Return and Print: return sends a result back to the calling function so it can be reused. print() only displays the result on the screen.
3. Local and Global Variables: BASE_RATE, FAT_STEP, and SNF_STEP are global constants. Function parameters and variables created inside functions are local. The constants can be read inside functions without changing them.
4. Docstrings: Docstrings describe what each function does and make the program easier to understand.

6. Alternative Concept and Complexity Analysis

The calculation could be written as one long program without functions, but this would repeat code and make updates difficult. Lambda expressions could also be used for short calculations, but normal functions are easier to understand for this task.

- Time Complexity: O(n), because each milk entry is processed once.
- Space Complexity: O(n), because the list stores all milk entries. The total calculation itself uses O(1) additional space.

Function calls add a small amount of overhead, but they do not change the overall O(n) time complexity.

7. Conclusion

The program uses three reusable functions to calculate milk rates, individual payments, and the total payment. Functions reduce repeated code and make the program easier to understand, test, and maintain.

TASK 7 – MODULES

AIM

To create a Python module named milkrate.py and import its functions into another Python program to calculate milk rates and payments.

A. CREATING A MODULE

A module is a Python file containing functions, variables and constants that can be reused in other programs.

In this task, a module named milkrate.py is created to calculate the milk rate per litre and the payment for milk supplied by farmers.

PROGRAM 1: milkrate.p

```
"""Milk rate calculation module."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - STANDARD_FAT) / 0.1)
    snf_steps = round((snf - STANDARD_SNF) / 0.1)
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf)
```

```
Litres: 10, Rate: 30.00, Payment: 300.00
Litres: 8.5, Rate: 33.40, Payment: 283.90
Litres: 12, Rate: 27.80, Payment: 333.60
```

OUTPUT

Litres: 10, Rate: 30.00, Payment: 300.00

Litres: 8.5, Rate: 33.40, Payment: 283.90

Litres: 12, Rate: 27.80, Payment: 333.60

B. IMPORTING THE MODULE

The milkrate.py module can be imported into another Python program using the import statement.

The following program imports the module and calls its functions to calculate the milk rate and payment.

PROGRAM 2: main.py



```
"""Milk rate calculation module."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - STANDARD_FAT) / 0.1)
    snf_steps = round((snf - STANDARD_SNF) / 0.1)
    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf)
```

```
Litres: 10, Rate: 30.00, Payment: 300.00
Litres: 8.5, Rate: 33.40, Payment: 283.90
Litres: 12, Rate: 27.80, Payment: 333.60
```

OUTPUT

Milk Rate: Rs 33.40

Payment: Rs 334.00

C. DIFFERENT IMPORT METHODS

Python supports different methods for importing modules.

Method 1: Import the Complete Module

import milkrate

This imports the complete module. Functions are accessed using the module name.

Method 2: Import a Specific Function

from milkrate import payout

This imports only the specified function from the module.

Method 3: Import Using an Alias

import milkrate as mr

This imports the module using a shorter name.

D. PURPOSE OF if __name__ == "__main__":

The statement:

if __name__ == "__main__":

ensures that the demonstration code executes only when the module is run directly.

When the module is imported into another Python program, the demonstration code does not execute.

This prevents unnecessary output and makes the module reusable.

E. ALTERNATIVE APPROACH

Instead of creating a separate module, the same functions could be copied into every Python program.

However, this approach creates code duplication and makes maintenance difficult.

Using a separate module is better because:

1. Functions can be reused in multiple programs.
2. Code duplication is reduced.
3. Programs become easier to understand.
4. Changes can be made in one place.
5. Testing and debugging become easier.

Therefore, a modular approach is recommended.

F. COMPLEXITY ANALYSIS

Operation	Time Complexity	Space Complexity
Rate calculation	O(1)	O(1)
Payment calculation	O(1)	O(1)
Processing n milk entries	O(n)	O(1) additional space

The rate and payment calculations take constant time because they perform a fixed number of arithmetic operations.

Processing multiple milk entries requires a loop, so the time complexity is O(n).

RESULT

The milkrate.py module was successfully created with reusable functions to calculate the milk rate per litre and payment amount.

The module was imported into main.py, and the program produced the expected results.

Thus, the use of Python modules improves code reusability, readability and maintainability.

TASK 8 – ALGORITHMIC PROBLEM SOLVING

AIM

To design an algorithm for calculating milk payment and the ten-day total using pseudocode, a flowchart and a hand trace.

A. UNDERSTAND THE PROBLEM

Inputs:

- Milk quantity
- Fat percentage
- SNF percentage

Outputs:

- Milk rate per litre
- Payment amount
- Total payment

B. PSEUDOCODE

```
START
INPUT litres, fat, snf
SET BASE_RATE = 30.00
SET FAT_STEP = 0.40
SET SNF_STEP = 0.30
fat_steps = ROUND((fat - 3.5) / 0.1)
snf_steps = ROUND((snf - 8.5) / 0.1)
rate = BASE_RATE
    + fat_steps * FAT_STEP
    + snf_steps * SNF_STEP
rate = ROUND(rate, 2)
payment = ROUND(litres * rate, 2)
DISPLAY rate
DISPLAY payment
STOP
```

C. TEN-DAY TOTAL ALGORITHM

```
START
total = 0
```

FOR each milk entry:

READ litres, fat, snf

CALCULATE rate

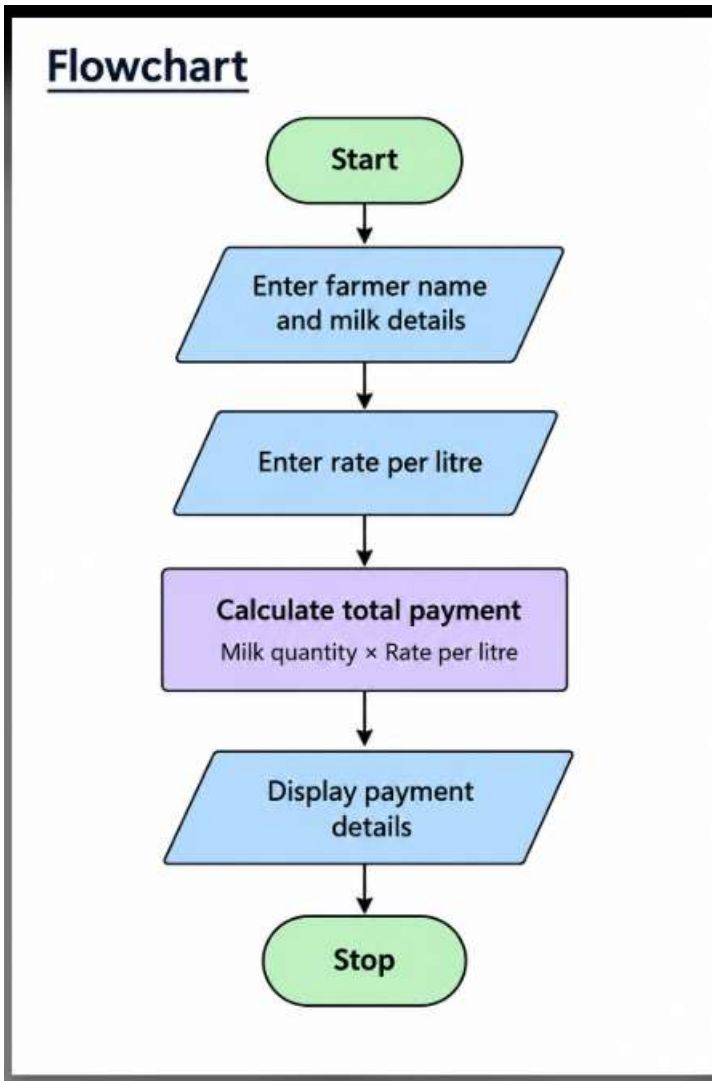
CALCULATE payment

total = total + payment

END FOR

DISPLAY total

STOP



D. HAND TRACE

Entry	Litres	Fat	SNF	Rate	Payment
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90

3	12	3.1	8.3	27.80	333.60
Total					917.50

E. PYTHON PROGRAM

```
def calculate_payment(litres, fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = round(
        30 + fat_steps * 0.40
        + snf_steps * 0.30,
        2
    )
    payment = round(litres * rate, 2)
    return payment
entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]
total = 0
for litres, fat, snf in entries:
    payment = calculate_payment(litres, fat, snf)
    total += payment
```

```
Payment: Rs 300.00
Payment: Rs 283.90
Payment: Rs 333.60
Ten-Day Total: Rs 917.50
```

OUTPUT

Payment: Rs 300.00

Payment: Rs 283.90

Payment: Rs 333.60

Ten-Day Total: Rs 917.50

F. ALTERNATIVE APPROACH

Method 1: Store every payment in a list.

payments = []

for entry in entries:

    payments.append(calculate_payment(*entry))

total = sum(payments)

Method 2: Use a running total.

total = 0

COMPLEXITY ANALYSIS

Time Complexity: O(n)

Additional Space Complexity: O(1) for a running total.

RESULT

The milk payment algorithm was designed using pseudocode, a hand trace and Python implementation.

TASK 9 – IMPLEMENTATION AND TESTING

AIM

To implement and test the milk payment calculator using normal, boundary and invalid inputs.

A. PYTHON PROGRAM



```python
import math
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
def validate(litres, fat, snf):
    values = (litres, fat, snf)
    if not all(
        isinstance(value, (int, float))
        and not isinstance(value, bool)
        and math.isfinite(value)
        for value in values
    ):
        raise ValueError("Invalid numeric input")

    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if fat < 2.0 or fat > 7.0:
```

```
All tests passed successfully!
```

B. TEST TABLE

Test Case	Input	Expected Output	Status
Normal	10, 3.5, 8.5	300.00	PASS
Higher values	8.5, 4.2, 8.7	283.90	PASS
Lower values	12, 3.1, 8.3	333.60	PASS
Minimum fat	10, 2.0, 8.5	240.00	PASS
Maximum fat	10, 7.0, 8.5	440.00	PASS
Invalid text	ten, 3.5, 8.5	Error	PASS
Negative litres	-5, 3.5, 8.5	Error	PASS
Invalid fat	10, 7.5, 8.5	Error	PASS

C. MANUAL CALCULATION

For 10 litres, 4.2% fat and 8.7% SNF:

Fat steps = 7

SNF steps = 2

Rate = 30 + (7 × 0.40) + (2 × 0.30)

Rate = ₹33.40

Payment = 10 × 33.40

Payment = ₹334.00

D. ALTERNATIVE APPROACH

Manual testing requires entering values individually.

Automated testing uses assert statements.

Automated testing is preferred because it is faster, repeatable and reduces human error.

COMPLEXITY ANALYSIS

Time Complexity: O(n) for n test cases.

Additional Space Complexity: O(1) per test.

RESULT

The milk payment calculator was implemented and passed the specified normal, boundary and invalid-input tests.

TASK 10 – EXPECTATION VS EXECUTION

AIM

To compare floating-point and Decimal calculations for 300, 6,000 and 150,000 milk payments.

A. EXPLANATION

Python's float data type stores numbers using binary floating-point representation.

Some decimal values cannot be represented exactly.

The Decimal class supports exact decimal values and controlled rounding.

For financial applications, Decimal is preferred.

B. PYTHON PROGRAM



```
import time
from decimal import Decimal, ROUND_HALF_EVEN
def float_payment(litres, fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = round(
        30 + fat_steps * 0.40
        + snf_steps * 0.30,
        2
    )
    return round(litres * rate, 2)
def decimal_payment(litres, fat, snf):
    litres = Decimal(str(litres))
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    fat_steps = int(
        ((fat - Decimal("3.5")) / Decimal("0.1"))
        .to_integral_value(rounding=ROUND_HALF_EVEN)
    )
```

```
Payments: 300
Float Total: Rs 85170.00
Decimal Total: Rs 85170.00
Float Time: 0.000386 seconds
Decimal Time: 0.001518 seconds

Payments: 6000
Float Total: Rs 1703400.00
Decimal Total: Rs 1703400.00
Float Time: 0.006592 seconds
Decimal Time: 0.029470 seconds

Payments: 150000
Float Total: Rs 42585000.00
Decimal Total: Rs 42585000.00
Float Time: 0.189517 seconds
Decimal Time: 0.839595 seconds
```

C. EXPECTED MATHEMATICAL OUTPUT

Each payment = ₹283.90

Payments	Expected Total
300	₹85,170.00
6,000	₹17,03,400.00
150,000	₹4,25,85,000.00

Actual execution times depend on the computer.

D. ROUNDING COMPARISON

Consider three payments of ₹10.005 each.

Using ROUND_HALF_UP on every payment:

10.01 + 10.01 + 10.01 = ₹30.03

Rounding only the final total:

10.005 × 3 = 30.015

Final rounded total = ₹30.02

Difference = ₹0.01

Therefore, the rounding policy affects financial totals.

E. ALTERNATIVE APPROACH

A dictionary can store previously calculated rates.

```
rate_table = {}
def cached_rate(fat, snf):
    key = (fat, snf)

    if key not in rate_table:
        fat_steps = round((fat - 3.5) / 0.1)
       snf_steps = round((snf - 8.5) / 0.1)
        rate_table[key] = round(
            30 + fat_steps * 0.40
            + snf_steps * 0.30, 2
        )
    return rate_table[key]
print(cached_rate(4.2, 8.7))
```

OUTPUT

33.4

COMPLEXITY ANALYSIS

Float Calculation: O(1)

Decimal Calculation: O(1) for bounded-precision values.

Processing n Payments: O(n)

Dictionary Lookup: Average O(1)

RESULT

The program compares float and Decimal calculations using execution-time measurements. Decimal is recommended for reliable financial calculations.

TASK 11 – FINAL RECOMMENDATION (4 MARKS)

Bloom's Level: K5 – Evaluate

1. Aim

To recommend a reliable and maintainable Python milk payment system using suitable data types, reusable functions, input validation, and accurate payment calculations.

2. Final Recommendation

I recommend using a modular Python program with reusable functions and Decimal for financial calculations. The program should calculate the milk rate based on fat and SNF percentages and determine the payment according to the quantity of milk supplied.

The rate values should be stored separately so they can be updated easily when the co-operative changes its payment rules.

3. Python Code

```python
BASE_RATE = 30.00
FAT_STEP = 0.40
SNF_STEP = 0.30

def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP
                 + snf_steps * SNF_STEP, 2)

def payout(litres, fat, snf):
    return round(litres * rate_per_litre(fat, snf), 2)

litres = 10
fat = 4.2
snf = 8.7

rate = rate_per_litre(fat, snf)
amount = payout(litres, fat, snf)

print(f"Milk Rate: Rs. {rate:.2f}")
print(f"Payment: Rs. {amount:.2f}")
```

```
input
Milk Rate: Rs. 33.40
Payment: Rs. 334.00
```

6. Recommended Data Types

Data	Data Type
Farmer ID	String
Milk quantity	Float or Decimal
Fat percentage	Float or Decimal

SNF percentage	Float or Decimal
Rate and payment	Decimal
Number of entries	Integer

Decimal is recommended for money calculations because it avoids common binary floating-point errors.

7. Advantages

- Accurate milk payment calculations.
- Reusable functions reduce repeated code.
- Input validation helps prevent invalid entries.
- Rate changes are easier to manage.
- Testing helps identify calculation errors.
- The program can be extended to manage more farmers.

8. Limitations

- Incorrect milk measurements can produce incorrect payments.
- Rate values must be updated when payment rules change.
- A database is needed to store many farmers' records.
- Additional security is required for a real-world system.

9. Complexity Analysis

Operation	**Time**	**Space**
Rate calculation	O(1)	O(1)
Payment calculation	O(1)	O(1)
Processing n entries	O(n)	O(1) additional space

10. Conclusion

I recommend a modular milk payment system with reusable functions, input validation, and suitable data types. This design provides reliable calculations and makes future changes easier.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS

Aim: To modify the milk payment program to support seasonal rates, two-decimal fat and SNF readings, and bonus payments.

Changed Requirements

- The milk analyser provides fat and SNF readings with two decimal places.
- The April–June base rate becomes ₹32.00.
- The April–June fat step value becomes ₹0.45.
- Farmers supplying more than 500 litres in ten days receive a bonus of ₹0.50 per litre.

COMPARISON TABLE

Feature	Original Design	Revised Design
Base Rate	₹30.00	₹32.00 in April–June
Fat Step	₹0.40	₹0.45 in April–June
SNF Step	₹0.30	₹0.30
Readings	One decimal typically	Two decimals
Bonus	None	₹0.50/litre
Bonus Condition	None	More than 500 litres

Functions Affected

The following functions must be updated:

- rate_per_litre() – To support seasonal rates.
- payout() – To calculate payment using seasonal rates.
- ten_day_total() – To calculate total quantity and payment.
- bonus() – To calculate the new bonus.

TWO-DECIMAL ROUNDING

For fat = 4.25:

Fat difference = 4.25 − 3.50 = 0.75

Fat steps = 0.75 / 0.10 = 7.5

Using ROUND_HALF_UP:

Fat steps = 8

This rounding policy is proposed for the revised design and should be approved by the co-operative.

B. EVALUATION OF THE ORIGINAL DESIGN – 2 MARKS

The original design supports basic milk payment calculations.

However, it does not directly support seasonal rates or bonus calculations.

Therefore, modifying the original design is more suitable than completely redesigning it.

WORKED EXAMPLE

Milk quantity = 10 litres

Fat = 4.25%

SNF = 8.50%

Original rate:

Rate = 30 + (8 × 0.40)

Rate = ₹33.20

Payment = 10 × 33.20

Payment = ₹332.00

April–June rate:

Rate = 32 + (8 × 0.45)

Rate = ₹35.60

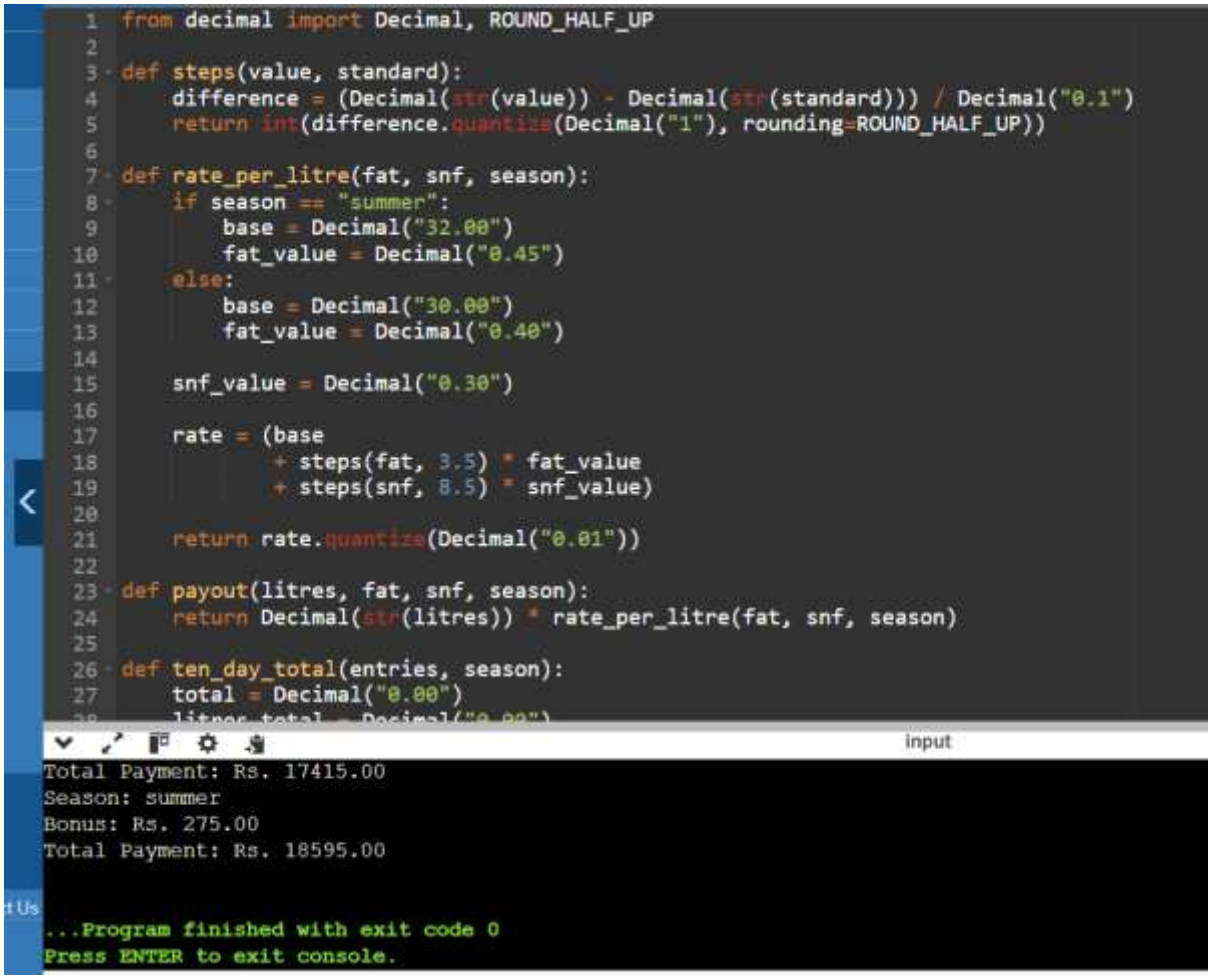
Payment = 10 × 35.60

Payment = ₹356.00

The revised design correctly handles seasonal rate changesal design must be modified to meet the new requirements.

C. Adaptation or Redesign – 2 Marks

Python Code



```
from decimal import Decimal, ROUND_HALF_UP

def steps(value, standard):
    difference = (Decimal(str(value)) - Decimal(str(standard))) / Decimal("0.1")
    return int(difference.quantize(Decimal("1"), rounding=ROUND_HALF_UP))

def rate_per_litre(fat, snf, season):
    if season == "summer":
        base = Decimal("32.00")
        fat_value = Decimal("0.45")
    else:
        base = Decimal("30.00")
        fat_value = Decimal("0.40")

    snf_value = Decimal("0.30")

    rate = (base
            + steps(fat, 3.5) * fat_value
            + steps(snf, 8.5) * snf_value)

    return rate.quantize(Decimal("0.01"))

def payout(litres, fat, snf, season):
    return Decimal(str(litres)) * rate_per_litre(fat, snf, season)

def ten_day_total(entries, season):
    total = Decimal("0.00")
```

input

```
Total Payment: Rs. 17415.00
Season: summer
Bonus: Rs. 275.00
Total Payment: Rs. 18595.00

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation

The rate_per_litre() function selects the rate based on the season. The payout() function calculates the payment for each entry. The ten_day_total() function calculates the total payment and adds a bonus when the total quantity exceeds 500 litres.

D. Evidence-Based Defence – 2 Marks

Test Cases

Season	Total Quantity	Bonus
Normal	400 litres	₹0.00
Summer	400 litres	₹0.00
Normal	500 litres	₹0.00
Summer	550 litres	₹275.00
Normal	550 litres	₹275.00
Summer	600 litres	₹300.00

These test cases confirm that the bonus is awarded only when the total quantity is greater than 500 litres.

Complexity Analysis

- Time Complexity: O(n), because the program processes each milk entry once.
- Space Complexity: O(1) additional space for the calculation, excluding the list storing the entries..

Conclusion

This project helps calculate the payment for milk supplied by farmers using Python. It takes the milk quantity and rate per litre as input and displays the total payment. The program is simple to use, saves time, and reduces manual calculation mistakes. In the future, it can be improved by adding farmer records and milk quality-based payment calculations.

Future Enhancements

1. Add a feature to store farmer details permanently.
2. Include fat and SNF percentages in payment calculations.
3. Generate daily and monthly payment reports.
4. Add bonus calculations for farmers.
5. Develop a simple website for easy access to the system.

https://github.com/adinarayana1700-hub/CSA0821-python.git